Protocol-Governed Systems: An Architecture for Deterministic Declarative Execution

Contact: bachipeachy@gmail.com ORCID Profile: https://orcid.org/0009-0007-3810-6520

Preface

This paper describes an architecture for software execution.

Most of what follows can be read without any prior knowledge of Protocol-Governed Systems. The architecture is general. It concerns the relationship between a description of behavior and the machine that carries that behavior out, and that relationship is the same whether or not one has ever heard of PGS. A reader interested in software architecture, and skeptical of any particular runtime, is exactly the reader this paper is written for.

Still, a few words of orientation will help, because Protocol-Governed Systems is the reference implementation in which the architecture was worked out and validated.

A protocol-governed system separates behavioral definition from behavioral execution. System behavior is described declaratively as executable protocols rather than embedded in imperative application code. A compiler resolves, validates, and compiles those declarations into a single authoritative artifact—the Protocol Snapshot. A runtime then executes that snapshot and nothing else. The runtime carries no behavioral knowledge of its own; every behavior it performs was determined, validated, and compiled before execution began.

> In a protocol-governed system, nothing runs that was not first declared, validated, and compiled into the Protocol Snapshot.

The companion to this paper—An Architecture for Closed-Loop Governed Transformation—took up the first half of that sentence: how a Protocol Snapshot comes to be declared, validated, and compiled. It established how governed transformation produces an executable baseline. It treated the second half—nothing runs that was not—as a promise that execution itself must preserve.

This paper keeps that promise. It asks what "a runtime then executes that snapshot and nothing else" actually requires: who is permitted to determine behavior, what authority belongs to the compiler versus the runtime, and what guarantees that execution introduces nothing the snapshot did not already contain.

The answer, like its companion's, turns on a single rule about authority. The compiler determines behavior; the runtime realizes it. That rule is not specific to PGS. It is an architectural principle for deterministic declarative execution.

Abstract

The runtime is not where behavior lives. It is where behavior is realized.

Software execution has stayed stubbornly imperative. For decades, construction moved toward the declarative — through higher-level languages, models, and generated code — but the runtime went on *deciding*. It branches on state, plans at dispatch time, constructs execution graphs on the fly, and infers what to do next. Behavior emerges *during* execution. When the runtime decides, it holds a share of behavioral authority, and that shared authority is the root of why runtime behavior is hard to replay, hard to audit, hard to verify, and bound to the platform that produced it.

This paper presents an execution architecture that partitions behavioral authority absolutely. It rests on a single rule — the **Execution Partition**: the compiler defines behavior; the runtime only realizes it. The rule is enforceable because the compiled **executable protocol** is *complete* — it requires no behavioral inference at run time — so the runtime is able to originate nothing. That completeness is the architectural bridge between the two papers of this pair: it is what the companion's transformation must produce, and what this paper's runtime consumes, realizing it without thought. Where the companion ends by making the protocol complete, this paper begins by depending on that completeness. From that single placement of authority the architecture's properties follow as consequences rather than goals: execution is **deterministic** (identical protocol, inputs, and initial state yield an identical trace), **replayable**, **auditable**, **portable** across runtimes, and **verifiable** without exhaustively running the system. The executable protocol is the enduring execution artifact; the runtime is a replaceable interpreter of it, standing to the protocol as a virtual machine stands to bytecode or a query engine to a query. The model depends on no particular runtime and on no particular language. Protocol-Governed Systems is the reference implementation in which it was worked out; a worked execution is given in an appendix so that the architecture in the body can be evaluated on its own terms.

Together with its companion, which shows how an executable baseline is *created*, this paper shows how that baseline is *executed*. The two are the halves of a single model: protocol-governed computing.

1. Positioning — an execution architecture, not a runtime

This is a companion paper. Its partner, *An Architecture for Closed-Loop Governed Transformation*, described how a business problem becomes a new executable baseline. This one describes how that baseline runs. Between them they intend to cover the whole of protocol-governed computing: one paper for how software comes to be, one for how it executes.

The five conceptual papers before the pair introduced the components: constitutional governance and the layered stack; the compiler that turns declarations into an admissible boundary; the runtime that executes that boundary without domain knowledge; the inversion that puts protocol before implementation; and the governed pipeline by which a protocol evolves. Among them is a *Runtime Conceptual Model* that describes the reference runtime in detail.

This paper is not that paper. It stands to the Runtime Conceptual Model as its companion stands to the earlier change-management work: it abstracts the *architecture* out of the reference implementation. It is, in the phrasing the companion used, more general than a runtime. A reader may accept the architecture and reject the runtime, or the reverse. Implementations age; architectures, if they are any good, do not. The mechanics of the reference runtime — its repositories, its command surfaces, its artifact formats — are confined to the appendices by design, and the Runtime Conceptual Model is cited there as the implementation-level treatment.

The claim this paper defends can be stated plainly:

> Behavioral authority can be partitioned so completely that the runtime originates none of it — and when it is, deterministic, replayable, portable execution follows by construction rather than by effort.

The paper is, at bottom, about *authority* — the same subject as its companion, one level down. The companion asked who may create *meaning*, and answered: only the human-directed author, never the machine. This paper asks who may determine *behavior*, and answers: only the compiler, never the runtime. That parallel is not decorative. It is why the two papers are one architecture rather than two — and it is compact enough to hold in a single view:

	Companion — Transformation	This paper — Execution
Concern	how a baseline is *made*	how a baseline is *run*
Authority partitioned	authority over **meaning**	authority over **behavior**
The one law	Knowledge Partition — the machine may not create meaning	Execution Partition — the runtime may not originate behavior
Where authority lives	with the human author	with the compiler, in the protocol
Enduring artifact	the governed transformation	the executable protocol
Core consequence	no drift (agreement by construction)	determinism and replay (behavior fixed at compile time)

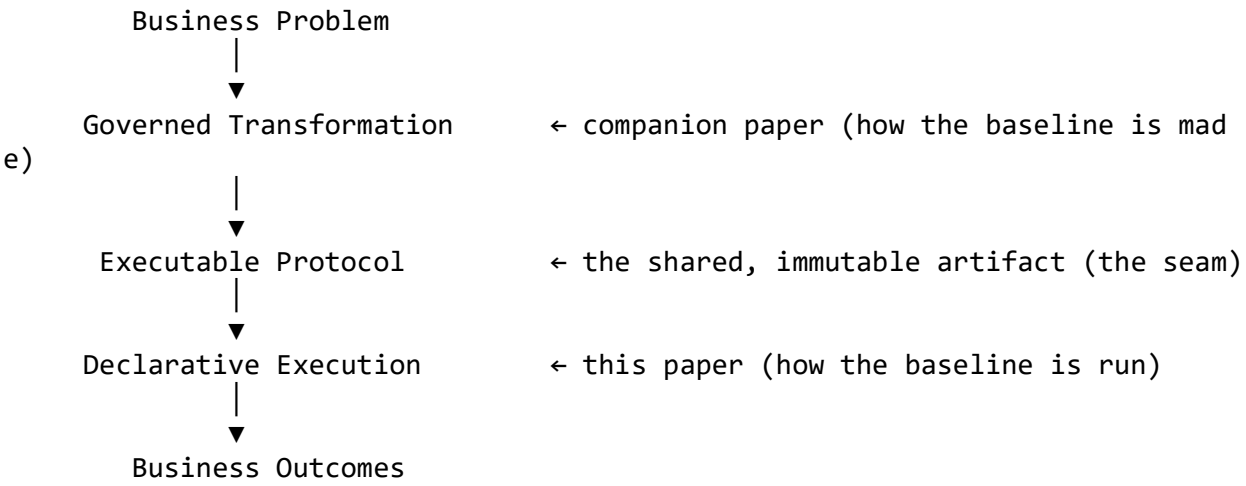
Five named ideas carry the argument, and it helps to have them in view from the start:

- the **Executable Protocol** (§4) — the immutable, complete description of behavior the runtime interprets;
- **Execution Closure** (§4) — the protocol needs no behavioral inference at run time;
- the **Execution Partition** (§5) — the compiler defines behavior; the runtime may only realize it;
- the **Runtime as Interpreter** (§6) — the runtime walks an already-decided structure and decides nothing;
- **Why the Executable Protocol Outlives the Runtime** (§11) — behavior belongs to the protocol, so the engine is replaceable.

The paper reaches them by following a single chain of questions, each section ending by posing the next. Why has execution stayed imperative (§3)? If behavior must not emerge at runtime, where does it live (§4)? What law keeps behavioral authority out of the runtime (§5)? If the runtime determines nothing, what is left for it to do (§6)? And how does execution proceed if the runtime decides nothing (§7)? A reader who loses the thread can re-find it here. For the skeptic who wants the full claim-set before the argument, §13 states the invariants as requirements any conforming implementation — PGS or not — must satisfy; a complete worked execution runs in Appendix D.

One picture carries the pairing, and both papers share it:

Figure 1 — Where this paper sits in protocol-governed computing.



Everything above the *Executable Protocol* is the subject of the companion; everything below it is the subject of this paper. The protocol itself is the seam they share, and §17 treats that seam honestly.

2. Context

The companion paper closed a loop. It showed how a business problem is transformed, in governed stages, into a new executable baseline — admitted, validated, promoted. But it treated the moment of execution as a given. When it needed to check that a candidate resolved its problem, it simply said the candidate was *run* and its behavior observed. It never asked what running requires.

That omission was deliberate, and it left a precise question beneath the whole companion. Its one-sentence summary for newcomers was: *nothing runs that was not first declared, validated, and compiled into the Protocol Snapshot.* The companion explained *declared, validated, compiled*. It did not explain *nothing runs that was not* — the clause that constrains the runtime. What discipline on the runtime makes that clause true? What stops a runtime, handed a complete snapshot, from quietly adding behavior of its own — a default here, an inferred branch there — and so reintroducing below the protocol exactly the drift the companion removed above it?

There is a further reason the question cannot be skipped, and it is worth naming because it makes the two papers interdependent rather than merely sequential. The companion's **Validation** stage — the one that decides whether a candidate actually resolves its problem — *runs the candidate*. It is already a consumer of the execution architecture this paper describes. So the companion did not merely defer execution; it silently depended on it. The architecture below is the machinery the companion's central guarantee stands on. §17 returns to this seam once both halves are on the table.

The question, then, is simple to state and consequential to answer. *Once an executable protocol exists, who has the authority to determine what it does when it runs?*

3. Why execution stayed imperative

Consider what nearly every runtime in wide use has in common.

A program is written. A runtime loads it. And then, as it runs, the runtime *decides* — it evaluates conditions, chooses branches, resolves polymorphism, schedules work, and often plans or constructs the very structure it is about to execute. The behavior of the system is not fully present before it runs; it is produced, moment by moment, by the runtime acting on state.

```
Program → Runtime → behavior emerges during execution
```

This is so normal that it is rarely seen as a choice. But it is a choice about *authority*. In this arrangement the runtime holds a share of the authority to determine behavior. It is not merely carrying out decisions already made; it is making decisions. And four familiar difficulties follow from that, none of them accidents of any particular language — just as the companion found four failures living in the specification's position in the process, these four live in the runtime's share of authority.

The first is **runtime decisioning**. Control flow is chosen dynamically, as a function of state the runtime observes. What the system will do is a property of the run, not of the artifact.

The second is **hidden behavior**. Because behavior is produced during execution, it cannot be fully known before execution. To learn what a system does, one runs it and watches — and even then one sees only the paths this run happened to take.

The third is **poor replay**. Re-running does not reliably reproduce, because the decisions depended on ambient conditions — time, ordering, external state — that are not part of the artifact and do not recur identically. Reproducing a past execution becomes an exercise in reconstructing an environment, not in replaying an artifact.

The fourth is **platform dependence**. Behavior is entangled with the engine that produced it. Move the program to a different runtime and its behavior may shift, because part of what it *does* lived in the runtime, not in the program. The behavior cannot travel, because it was never wholly in the thing that travels.

None of these are failures of engineering skill. They are consequences of *where behavioral authority sits*. As long as the runtime holds a share of it, behavior emerges at run time, and everything that follows from emergence — opacity, non-replayability, platform lock-in — follows too.

So the authority has to move. Which raises the question of where it should go.

4. The executable protocol: the locus of behavior

If behavior must not emerge at runtime, then it must already exist, complete, before runtime begins. The thing that holds it is the central object of this paper, and it deserves to be named exactly.

> **Executable Protocol.** An immutable, complete, executable description of system behavior that requires no behavioral inference during execution.

Every word is load-bearing. *Immutable*: it does not change as it runs; execution reads it and never writes it. *Complete*: it contains everything needed to execute — no piece of behavior is left to be supplied later. *Executable*: it is not a description *of* a program to be built, but the very thing the runtime carries out. And *requires no behavioral inference during execution*: at no point must the runtime work out what was meant, choose among unstated options, or fill a gap with a default.

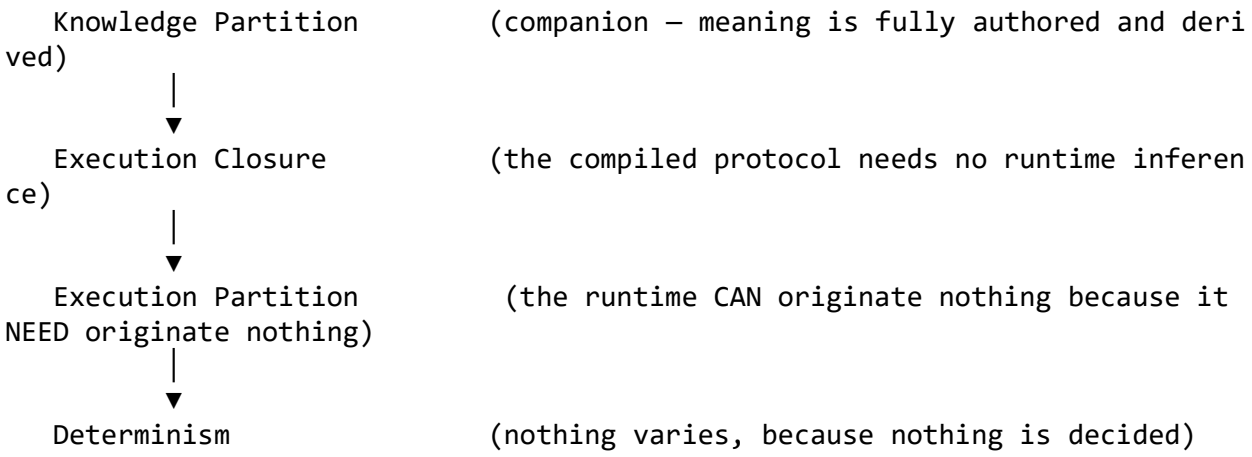
It helps to say what the executable protocol is *not*. It is not source code: nothing is generated from it at run time, and there is no compilation step left to perform. It is not configuration: it is complete rather than parametric, a whole behavior rather than knobs on one. And it is not data: it is behavioral, a description of what happens rather than of what is. The nearest familiar analogue is compiled bytecode, and §11 develops that analogy; but even bytecode is executed by a machine that still makes runtime decisions, and the executable protocol is designed so that it need not.

Where does it come from? For readers who have not read the companion, one sentence closes the gap: the executable protocol is the output of compilation — the compiled Protocol Snapshot the companion's transformation produces. This matters here only for what it implies about authority, and §5 draws the implication. The point to carry forward is that some earlier, governed process *determined* this behavior, in full, before any runtime saw it.

That completeness has a name, and it is the quiet center of the whole architecture:

> **Execution Closure.** Everything required to execute is present in the protocol. The runtime needs no external knowledge, no inference, and no default.

Execution closure is the execution-side twin of the completeness the companion's progressive semantic enrichment drives toward: there, no stage may leave meaning to be invented later; here, no protocol may leave behavior to be invented at run time. And it is what makes the next section's rule *enforceable* rather than merely aspirational. One can forbid the runtime to decide only if the runtime never *needs* to decide — only if nothing it will encounter is left open. Execution closure guarantees exactly that. It is the bridge between the two papers: the companion's discipline produces a protocol so complete that the runtime can be forbidden to think.



We now have an object that holds behavior in full, and a property — execution closure — that makes it possible to keep the runtime out of behavior entirely. What is the rule that does so?

5. The Execution Partition

Here is the rule on which everything else turns, and this section makes only this one point, because the point is strong enough to stand alone.

Because the executable protocol is the output of compilation, and because execution closure means the runtime never needs to infer anything, *the compiler owns behavioral authority in full, and the runtime owns none of it.* Name this the **Execution Partition**, and state it, as its companion states the Knowledge Partition, by what is partitioned — behavioral authority — not by where it applies:

> **Execution Partition.** Behavioral authority is partitioned absolutely. The compiler defines behavior; the runtime realizes it. Knowledge belongs to authors; behavior belongs to protocols.

Stated as a law, with its corollary drawn out separately, in the manner of mathematics:

> **Architectural Law.** The runtime may *execute* behavior but may never *originate* it.
>
> **Corollary.** Therefore nothing is decided at runtime that was not decided during compilation.

The law and the corollary are different claims, and keeping them apart is what makes the architecture rigorous rather than slogan-shaped. The law is about *authority* — who is permitted to determine behavior. The corollary is about *time* — when behavior is fixed. The corollary follows from the law: if the runtime may originate no behavior, then every behavior must already have been determined by the only participant that may, the compiler, and that participant runs before the runtime does.

Readers of the prior series will recognize the partition's embryo. The Runtime Conceptual Model stated it as a maxim: *the compiler governs possibility; the runtime governs realization*. This paper's contribution is to promote that maxim to the architecture's organizing law — to name what it partitions, state it with its corollary drawn out, and derive the rest of the execution architecture from it as consequences. The law was discovered in the reference implementation before it was named; that is the usual order of discovery in this series, and the right one.

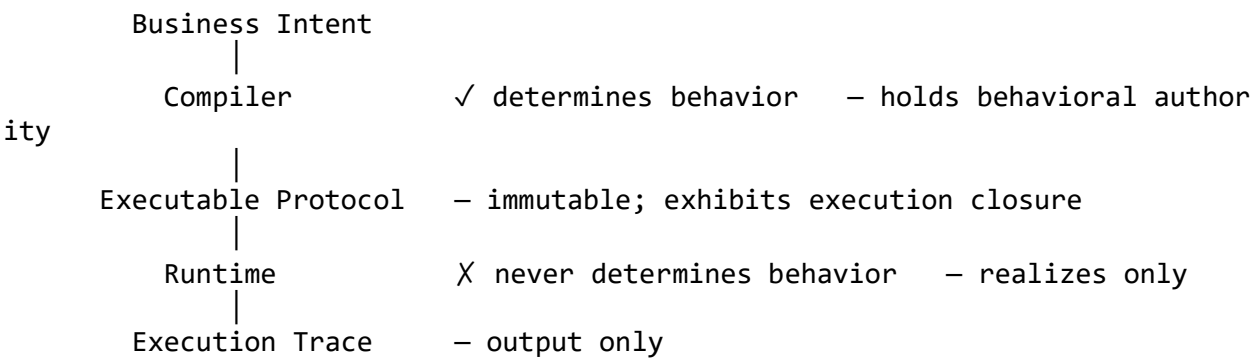
The reason the runtime is *forbidden* to originate behavior — not merely discouraged — is the same reason the companion forbids the platform to create meaning. Any behavior the runtime originates is behavior that escaped compile-time governance: it was never validated, never given provenance, never admitted. It is precisely the drift the companion abolished above the protocol, reappearing below it. A runtime that decides anything at all reopens the gap the whole architecture exists to close. The only honest position is that the runtime decides nothing.

From the law, a list of prohibitions follows directly. Under the Execution Partition, the runtime does none of the following:

- no runtime planning;
- no runtime graph construction;
- no runtime semantic interpretation;
- no runtime discovery;
- no runtime branching on domain logic;
- no runtime fallback — no defaults, no retries of its own devising, no degraded modes;
- dispatch only; protocol interpretation only.

Each of these is a way a conventional runtime originates behavior, and each is ruled out not by policy but by the placement of authority. The runtime cannot plan, because planning is deciding. It cannot construct the execution graph, because the graph was constructed at compile time. It cannot interpret meaning, because meaning is not its to interpret. And it cannot fall back, because a fallback is a behavior nobody declared — an item worth emphasis, because fallback is the most common way real runtimes originate behavior in practice. What is left is realization: carrying out, faithfully, a behavior determined entirely elsewhere.

Figure 2 — The authority split.



(The paper is ASCII-diagrammed; rendered versions of Figures 1 and 2 belong in final typesetting, in the style of the companion's figures.)

That single rule — one holder of behavioral authority, and it is never the runtime — is the foundation the rest of the architecture is built on. Taking it seriously raises the obvious next question. If the runtime determines nothing, what is left for it to do at all?

6. The runtime as interpreter

The runtime, it turns out, has plenty to do. It simply does none of it in the register of *deciding*. It is worth being exact about the division, because the exactness is the architecture.

The compiler and the runtime own disjoint sets of responsibilities, and the line between them is the Execution Partition made concrete:

- **The compiler owns:** semantic resolution, dependency resolution, graph construction, validation, optimization, and the generation of the protocol itself. Every act that determines *what the system does* belongs here.
- **The runtime owns:** verification of the protocol's integrity, interpretation of the graph, scheduling, dispatch, state transitions, and trace emission. Every act that *carries out* an already-determined behavior belongs here — and nothing else does.

The first of those responsibilities deserves a sentence of its own, because it is the enforcement of the partition rather than a housekeeping step. The interpreter's first act, before any traversal, is **verification**: it confirms that the protocol it holds is exactly what the compiler produced and attested. On a mismatch it refuses to execute — no override, no warning, no degraded start. An unverified protocol would be a path by which behavior enters the system that neither the compiler nor the runtime originated, and the partition admits no such path.

The runtime, in this architecture, is a **protocol interpreter**, not a program executor. The distinction is not pedantic. A program executor is handed instructions and, in carrying them out, makes decisions the instructions leave to it. A protocol interpreter is handed a structure in which every decision has already been made, and its whole task is to walk that structure faithfully. Every edge the interpreter could take was declared and validated before it began; the interpreter chooses among them not by judgment but by reading, from the protocol, which edge a given outcome selects. Henceforth this paper uses *runtime* and *interpreter* interchangeably for this component, whose sole function is the faithful realization of the protocol; where the shift in word carries emphasis, it is toward the interpretive character the name *interpreter* makes explicit.

There is a deliberate consequence of this that runs against a reader's instinct: in a paper about execution, the runtime is intentionally the *least* interesting component. It is meant to be boring. The hero is the protocol. A runtime one can describe in a paragraph — interpret, schedule, dispatch, record — and replace without touching behavior is exactly the runtime the architecture wants. The brilliance, such as it is, was spent at compile time and now sits inert in the protocol, waiting to be realized. The interpreter adds nothing to it and takes nothing from it.

If the runtime only walks a structure, we should look at how the walk proceeds — at what "declarative execution" actually means when no step involves a decision.

7. Declarative execution: traversal without decision

Execution, under this partition, is traversal. A business input arrives; it is normalized and admitted; a workflow is traversed; capabilities are dispatched; a result is projected out; a trace is written. At no step is behavior invented. The path is a reading of the protocol, not a computation over it.

The altitude of the thing, setting aside the reference implementation's particulars, looks like this:

```
Protocol → Workflow → Capability → Contract → Execution
```

For readers of the series, this flow is made concrete in the reference implementation as a set of nine declared *execution concerns* — from transport ingress that normalizes an external call, through admission and the workflow topology, to the capabilities that compute or effect, and the egress that projects the result. Each concern has one role, and the runtime treats each as compiled data; the body keeps to the abstract flow, and Appendix B carries the nine.

A **workflow** is the unit of orchestration, and it is where the architecture most sharply departs from convention. In a conventional system, orchestration is procedural code: the runtime evaluates conditions and branches. Here a workflow is a **governed directed graph**, and its traversal is determined by each node's *declared outcome*. A capability runs and reports one of its enumerated outcomes; that outcome selects the next node, according to routing the compiler fixed in the protocol. The runtime performs no branching logic of its own. It does not decide where to go; it reads where to go. Routing is data, not logic — determined once, at compile time, and merely followed at run time.

The same discipline governs how a step's inputs are found. A capability does not search for its inputs or infer them from context; it receives them through declared references resolved against results the traversal has already produced. Nothing is looked up by heuristic. (The concrete resolution mechanism — how a reference names a prior result — is a reference-implementation detail and is left to Appendix B.)

This is what "declarative execution" means precisely: not that the system is configured rather than coded, but that **no step of execution involves originating behavior**. There is no code generation, no planning, no semantic inference anywhere in the traversal. Each step is the faithful realization of a behavior the protocol already fixed.

Two further disciplines follow from traversal-without-decision and are worth naming. The first is **transport neutrality**: the topology cannot tell — and is forbidden to learn — how an invocation arrived. A workflow that could detect its transport could behave differently per caller, which is origination by another name; so the boundary normalizes every ingress to one canonical form, and the topology sees only that form. The second is **structural parallelism**: because executions share no ambient state, pure capabilities are unconditionally parallelizable, and traversal advances only on declared outcomes, the topology itself declares what is independent. Concurrency is not engineered into the

runtime; it is read off the protocol, and the substrate exploits that independence to whatever degree it can. Neither discipline is an optimization. Both are the partition, seen once at the boundary and once across executions.

And what of failure? A side-effecting capability can meet a network that is down or a store that refuses; a payload can be malformed; an artifact can be missing. In a conventional runtime this is exactly where origination concentrates — retries, fallbacks, defaults, degraded modes, all invented at the moment of need. Here the answer is the same as everywhere else: failures are **declared outcomes**. A capability's contract enumerates its failure outcomes alongside its successes, and the routing for each was fixed at compile time; the negative path is as declared as the positive one. And when execution encounters what the protocol does not answer — a missing artifact, an undeclared condition — the runtime **refuses**. It does not improvise, default, or degrade; it halts and reports. Refusal is best understood not as an origination of behavior but as the *enforcement of the protocol's boundary*: it is the runtime's one and only governance function, and it exists precisely to prove that the protocol is a closed, complete world. Where the protocol answers, the runtime realizes; where it does not, the runtime refuses. In neither case does it decide.

Traversal without decision has a striking consequence, and it is the property the whole architecture is usually valued for. If the runtime decides nothing, what can be said about the *outcome* of a run?

8. Determinism, replay, and the trace

If behavioral authority sits entirely with the compiler, then a run has nothing left to vary. The protocol is fixed; execution closure means no inference intervenes; the runtime originates nothing. So the same protocol, the same inputs, and the same initial state produce the same outputs — and, because the runtime records what it did as it did it, the same **trace**.

The initial state belongs in that list and should not be smuggled past. Side-effecting capabilities read governed stores, so the state of those stores is part of what an execution is a function of: an append against an empty chain and an append against a chain whose head has moved are different inputs to the function, and they deterministically take different declared paths. Determinism here means exactly this — execution is a pure function of protocol, inputs, and initial state, with nothing else, no ambient condition and no runtime choice, anywhere in the function. So a second, identical invocation against a changed store will *not* produce an identical trace — and this is not a crack in determinism but a demonstration of it: the second run follows the declared path for its new initial state, exactly as the first followed the path for its own. Appendix D shows both faces of the claim on a real execution: strict replay when the state is held fixed, and the declared negative path, just as deterministic, when it is not.

It is worth being exact that determinism here is not a feature the runtime works to provide. A conventional runtime that wants determinism must *suppress* sources of variation it would otherwise have — must be careful about ordering, about clocks, about ambient state. This architecture has no such sources to suppress, because it granted the runtime no authority from which variation could arise. Determinism is not engineered; it is what remains when the runtime is permitted to decide nothing. It is a consequence of the Execution Partition, not an addition to it.

The **trace** deserves its own emphasis, because it inherits a discipline from the wider PGS model: the trace is *output only*. It is written by the runtime as a record of execution and is never read back as input by any component — not by the compiler, not by the protocol, not by a later run. A trace is a consequence of execution, never a cause of it. Because execution is deterministic, the trace is a faithful and complete account: everything the system did is in it, and nothing the system did depended on anything not derivable from protocol and inputs.

From determinism and an output-only trace, three payoffs follow, and each is a property the imperative runtime of §3 could not have:

- **Replay.** A past execution can be reproduced exactly by re-running the same protocol on the same inputs against the same initial state. Reproduction is replaying an artifact, not reconstructing an environment.
- **Audit.** The trace is the whole truth of what happened. To know what a system did, one reads its trace; there is no hidden behavior that escaped the record, because there was no behavior the runtime originated off the books.

- **Verification and testing.** Behavior can be checked without the ambiguity that non-determinism imposes. A test that passes once passes always, for the same inputs, because there is nothing left to differ between runs.

One further property of the trace is worth carrying up from the reference model: traces are *substrate-neutral*. Two conforming runtimes on radically different substrates, executing the same protocol on the same inputs and state, produce semantically equivalent traces — the same artifacts, the same paths, the same outcomes, differing only in substrate-local detail such as timestamps. Audit therefore does not depend on access to the substrate that produced the evidence; the trace, checked against the protocol, *is* the evidence.

This last payoff is the one the companion quietly relied on. Its Validation stage checks a candidate's observed execution against a declared outcome — a check that is only meaningful if execution is deterministic and its trace is faithful. The companion's central guarantee, in other words, rests on this section. §17 makes that dependence explicit.

Determinism concerns behavior. But execution also touches *state* — and a natural worry arises. If the runtime maintains state, does it not, through state, hold some authority after all?

9. State as a governed artifact

The worry is worth taking seriously, because state looks like the place where a runtime might smuggle authority back in. If the runtime owns the data, perhaps it owns some behavior through the back door.

It does not, and the reason is that the runtime does not own the state either.

> Protocols govern state; the runtime merely maintains it.

The stores that hold a system's data, their topology, and their ownership are all *protocol-defined*. Which entities are stored, where, under whose ownership, and what transitions are permitted — these are declarations in the protocol, not decisions in the runtime. And the point is sharper than custody: a **state transition is itself declared behavior**, governed exactly as control flow is. When a side-effecting capability writes to a store, the write it performs, the store it targets, and the conditions under which it is permitted were all fixed and validated at compile time. The runtime does not decide *that* the state changes, or *how* it changes, any more than it decides which node comes next; it realizes a transition the protocol already determined. The runtime has no more authority over a change to state than it has over a change to control flow — which is to say, none. It holds the data and applies the declared transitions, and it has no authority to decide what the data means or how it may change.

This is the same partition seen from a third angle. The companion said the machine may not create *meaning*. This paper has said the runtime does not own *behavior*. It now adds that the runtime does not own *state*: storage topology is a governance concern, resolved at compile time, exactly as behavior is. The runtime is stripped, deliberately, of every kind of authority — over meaning, over behavior, over state — and what is left is pure realization. (The concrete store layout and the way ownership is declared are reference-implementation matters, left to Appendix B.)

If the runtime dispatches capabilities but has no authority over what they mean or do, we should look closely at the capability itself — the unit the runtime actually invokes — and at exactly how little the runtime is permitted to know about it.

10. Capabilities as governed execution units

A **capability** is the unit of execution the runtime dispatches, and it is governed the same way everything else is: by a declared **contract**. The contract states the capability's inputs, its outputs, and its enumerated outcomes. That contract is the *entire* interface between the runtime and the capability. The runtime knows the contract and knows nothing else — not what the capability means, not how it is implemented, not what it does beyond producing one of its declared outcomes.

This deliberate ignorance is a feature, and it is the execution-side echo of a property the companion prized. The companion made *workers* — the sources of judgment that author a transformation — interchangeable, because each presented the same governed interface regardless of whether it was a person or a model. Here, *implementations* are interchangeable beneath a contract, because the runtime sees only the contract. One implementation of a capability may be swapped for another that satisfies the same contract, and the runtime cannot tell and does not care. The behavior the runtime realizes is defined by the contract, which the compiler governs; the code beneath is realization, not authority.

An objection arrives here, and it deserves the direct answer the companion gave its twin. If implementations are conventional code, has behavior not simply moved *into* the code — the partition preserved in name and lost in fact? The answer has three parts, and honesty requires all three. First, the contract bounds what an implementation can be: its inputs, outputs, and outcomes are fixed, and conformance to the contract is compiler-checked, so an implementation cannot widen, reroute, or re-interpret the behavior it fills. Second, the compiler does not — and does not claim to — guarantee that a conforming implementation is behaviorally *correct*; a capability can satisfy its contract and still compute the wrong thing. That residue is real, and it is judged by the companion's Execution Validation stage, which runs the realized candidate against the outcome its business problem declared. Third, between the two — conformance bounding the shape, validation judging the behavior — no *ungoverned* behavior survives in the code. What the implementation contributes is effort, not authority; the authority over what it must do sat upstream all along.

The distinction between kinds of capability — those that are pure computations and those that touch external state — matters to the architecture but is developed in the reference implementation rather than the body; Appendix B carries it. What matters here is only the interface: the runtime dispatches against contracts, and a contract carries no behavioral authority the compiler did not put there.

We have now stripped the runtime of authority over behavior, over state, and over the meaning of the capabilities it dispatches. A question presents itself that turns this litany of prohibitions into the paper's strongest positive claim. If the runtime holds none of the behavior, is it even necessary that *this* runtime be the one that runs the protocol?

11. Why the executable protocol outlives the runtime

It is not. And this is the architecture's strongest contribution after the partition itself.

Because the executable protocol holds behavior in full and the runtime holds none, the runtime is *replaceable*. A different runtime — differently written, on different hardware, in a different language — that faithfully interprets the same protocol produces the same behavior, because the behavior was never in any runtime to begin with. The protocol is the asset; the runtime is a consumable.

The familiar analogues are worth invoking, carefully and without claiming equivalence. A Java virtual machine executes bytecode, and the bytecode outlives any particular JVM: the same class file runs on implementations written decades apart, by different vendors, on machines its authors never saw. A SQL query is executed by an engine, but the query belongs to no engine; the same query runs on many. In each case the durable thing is the description, and the engine is interchangeable beneath it. The executable protocol is to a protocol-governed runtime what bytecode is to a JVM — with one difference that sharpens the point: bytecode is executed by a machine that still makes runtime decisions, whereas the executable protocol is complete enough (execution closure) that its interpreter need make none. It is, if anything, a purer instance of the same idea.

The long-term value of this is hard to overstate and easy to underappreciate. The runtime may be rewritten for a faster substrate, ported to a new platform, or replaced wholesale when its language falls out of favor — and none of it touches behavior. The behavior sits in the protocol, unmoved, because it was never entrusted to the engine. This is the sense in which the title of this section is literal rather than rhetorical: the protocol outlives the runtime because the runtime never held anything the protocol needs.

The architecture is now fully in view. Before collecting its invariants, it is worth refusing four misreadings — each a place where the Execution Partition's absoluteness could be mistaken for a narrower or more rigid claim than it makes.

12. What the Execution Partition does not mean

A rule as uncompromising as the Execution Partition invites misreadings, and four are worth naming and setting aside before the summary, because each mistakes the removal of the runtime's *authority* for the removal of something else.

It is not a rejection of state. The architecture governs state; it does not eliminate it. Systems built this way hold registries, ledgers, and evolving records, and are expected to. What is removed is not state but the runtime's authority *over* state: the transitions are declared and validated at compile time (§9). A stateless system is not the goal — a governed-state system is.

It is not a claim that execution is static. Determinism is not rigidity. A system built this way responds to a changing world — to new inputs and evolving state — and responds *differently* as they change; it simply responds *deterministically*, along paths the protocol declared. A second invocation against a changed store deliberately takes a different path (§8, Appendix D). Determinism constrains how behavior is decided, not whether the system can react.

It is not a performance model. This is an architecture for correctness and determinism, not for speed. How fast an interpreter runs, how it schedules, how far it exploits the parallelism the topology declares — these are implementation concerns of the interpreter, and they vary by substrate exactly as they should. The architecture fixes *what* executes and *who may decide it*, and says nothing about how quickly.

It is not a programming model. The Execution Partition governs how behavior is *executed*, not how capability code is *written*. A capability's implementation may be written in any language and any paradigm; the runtime sees only its contract (§10). The architecture prescribes how behavior is governed and realized, not how the realizing code is written.

Each misreading substitutes some other loss for the one the architecture actually makes. It removes the runtime's authority to originate behavior, and nothing else — not state, not responsiveness, not performance, not the freedom to implement. With those set aside, the invariants can be stated plainly.

13. Architectural invariants

The architecture can be stated as a small set of invariants, each one a consequence of the Behavior Partition and each one mechanically checkable. Scattered through the argument, they are easy to lose; gathered here, they are easy to cite.

- **Runtime originates no behavior.** Every behavior the system exhibits was determined at compile time.
- **The executable protocol is immutable.** Execution reads it and never writes it.
- **The runtime executes only a verified protocol.** Before any traversal, the protocol is confirmed to be exactly what the compiler produced and attested; on mismatch the runtime refuses.
- **The execution graph is complete before runtime.** Traversal follows a structure fixed at compile time; none is constructed during the run (execution closure).
- **The runtime performs no semantic inference.** Nothing is guessed, defaulted, or interpreted at run time.
- **The runtime fails hard.** What the protocol does not answer, the runtime refuses; there is no fallback, no default, and no degraded mode.
- **Traces are output only.** They are written by the runtime and never read back as input by any component.
- **Contracts define execution interfaces.** The runtime dispatches against declared contracts and knows nothing of implementations beneath them.
- **State topology is protocol-governed.** Stores and their ownership are declared, not chosen by the runtime.

Read one way, these are properties this architecture happens to have. Read the other way — the more useful one for anyone who wants declarative execution without PGS — they are **requirements**. An implementation is an instance of this architecture, whatever its technology, if and only if it honors all nine. A realization that honors them is a declarative-execution architecture; one that violates any of them is not, whatever it is called. This is the exact sense in which the architecture is separable from its reference implementation: the invariants say what must be true, not how any runtime makes it true.

For readers of the prior series, these invariants gather what the Runtime Conceptual Model published as observed properties of the reference runtime — implementation independence, hosting transparency, projection independence, runtime multiplicity, transport orthogonality, structural parallelism, runtime stability, and trace portability — and restate their common root. Each of those properties is a face of some invariant here: implementation independence and runtime multiplicity follow from behavior living wholly in the protocol; transport orthogonality is the neutrality of §7; structural parallelism is topology-declared independence; trace portability is the substrate-neutral evidence of §8. That paper described the properties as found in the reference implementation; this one derives them from the law the implementation was obeying.

From these invariants a set of consequences follows — closely enough that they are worth stating as theorems.

14. Architectural theorems

These are not design goals. They are logical consequences of the architecture.

The invariants of §13 are premises. The properties the architecture is valued for are their consequences, and because each follows from the premises rather than being asserted alongside them, they are fairly called theorems. They are stated plainly, not as marketing, and each can be checked against the argument that precedes it.

- **Determinism.** Given the same protocol, inputs, and initial state, execution produces the same outputs — because the runtime originates no behavior and infers nothing, so nothing can vary.
- **Replayability.** Any execution can be reproduced exactly — because it is a deterministic function of protocol, inputs, and initial state, and the trace records them faithfully.
- **Portability.** The protocol runs identically on any conforming runtime — because behavior lives in the protocol, not the engine.
- **Auditability.** The trace is a complete account of what happened — because no behavior was originated off the record.
- **Verifiability.** Behavior can be checked without exhaustive runtime exploration — because it is fixed and closed before the run, and can be examined in the protocol.
- **Composability.** Executions compose — because each is a pure function of protocol, inputs, and initial state, with no hidden runtime contribution to entangle them.
- **Governability.** Execution is governed end to end — because no ungoverned runtime decision is left anywhere for behavior to escape through.
- **Securability.** What the runtime cannot know, an attacker cannot exploit through it — a runtime with no routing logic of its own cannot be manipulated into alternate routes, and one with no evaluation path for free text cannot be injected through it. The runtime's ignorance is a security property.

Each theorem is a payoff the imperative runtime of §3 could not obtain, and each traces back to the same premise: behavioral authority was placed entirely with the compiler and denied entirely to the runtime.

15. Why the executable protocol endures

There is a deeper reason to place behavioral authority where this architecture places it, and it is worth drawing out on its own, because it is the payoff of the abstract's central claim — that the executable protocol, not the runtime, is the thing that endures.

Consider what a conventional running system actually is: a stack of mortal layers. A program sits on a framework, which sits on a language, which sits on a runtime, which sits on an operating system and hardware. Every layer has a finite life. Runtimes are rewritten; languages fall out of fashion; frameworks are abandoned; hardware is retired. And because behavior is smeared across these layers — some of it in the program, some of it produced by the runtime — when a lower layer changes, the behavior above it is at risk. The system's behavior is only as durable as the least durable layer that holds a piece of it.

This architecture concentrates behavior in one layer and makes that layer the durable one. The executable protocol holds behavior in full; every layer beneath it — the runtime, its language, the hardware — holds none. So every layer beneath is replaceable without loss. Replace the runtime, and the same protocol produces the same behavior on the new one. Port to new hardware, and nothing about behavior moves, because behavior was never in the hardware's keeping. What must persist is only the protocol, because the protocol is the only layer that was ever entrusted with behavioral authority.

This is the precise sense in which an architecture, unlike an implementation, does not age. A reader years from now, on runtimes and hardware not yet built, could take the executable protocol and run it faithfully, because everything the system does was placed in the protocol and nothing was left to the engine. The runtime is designed to be forgotten. The protocol is designed to last.

> The enduring execution artifact is the executable protocol; the runtime is merely its deterministic interpreter.

16. Related work

The architecture has neighbors, and naming them precisely locates the contribution. In each case the neighbor shares an intuition with this work but stops short of the Execution Partition — either by permitting runtime decisioning or by lacking a single compile-time behavioral authority.

Virtual machines and bytecode. The JVM and CLR separate a portable, compiled description from the engine that runs it, and the portability of bytecode is a genuine forerunner of what this paper calls runtime independence. But a virtual machine still makes runtime decisions — dynamic dispatch, just-in-time planning, runtime type resolution — so behavioral authority remains shared. The executable protocol pushes the idea to its limit: the description is complete enough that the engine decides nothing.

Workflow and business-process engines. BPM systems execute declared process graphs and are, in form, close to the workflow traversal of §7. But most permit expressions, scripts, or gateways evaluated at run time, so behavior is again partly produced by the engine. The Execution Partition forbids exactly this: routing is data the compiler fixed, never logic the engine runs.

Declarative infrastructure. Infrastructure-as-code and reconciliation-loop systems — the Terraform and Kubernetes lineage — declare a desired state and let an engine converge toward it. The declarative spirit is shared, but convergence is a runtime activity: the engine plans and decides how to reach the declared state. Here there is no convergence to plan, because the execution graph is complete before the run.

Database query execution. A query engine executes a declarative query, and the separation of query from engine is a strong parallel to protocol from runtime. But the engine plans — it chooses an execution strategy at run time. The executable protocol carries its own resolved structure, so no run-time planning remains.

Rules engines. Declarative rule systems are the sharpest near-miss of all: the rules are declared, but a matching engine evaluates them against working memory at run time, deciding which rules fire and in what order. Declared rules with a deciding engine is precisely the arrangement the Execution Partition forbids — the declarations are data to a runtime that still originates the behavior of applying them.

Functional and declarative programming. The tradition that asks programs to describe *what* rather than *how* — running back to Backus's critique of the von Neumann style — is the philosophical ancestor of this work. This paper's contribution is not the declarative stance itself but the governance around it: a single compile-time authority, execution closure, and an interpreter forbidden to decide.

None of these is claimed as equivalent, and none is dismissed. The architecture is best understood as the point where several long-running intuitions — portable descriptions,

declared orchestration, separation of description from engine — are taken together to their common limit by one rule about authority.

17. Relationship to the transformation architecture

The two papers meet at a single object, and it is time to be honest about how.

The **executable protocol** — the compiled baseline — is the shared, immutable artifact where the companion ends and this paper begins. The companion produces it; this paper consumes it. Figure 1 drew the handoff as a straight line, and at the level of the artifact it is one: a transformation yields a protocol, and a runtime executes it.

But the seam is not a clean one-way handoff, and pretending otherwise would misdescribe the architecture. The companion's **Validation** stage — the stage that decides whether a candidate actually resolves its business problem — *runs the candidate*. It is already a consumer of the execution architecture this paper describes. So the execution model is not merely downstream of the transformation model; the transformation model's central guarantee *depends on* it. The two interlock the way a compiler and a runtime interlock: neither is wholly prior, because each reaches into the other. The companion needs execution to validate; execution needs a compiled protocol to run.

Read together, the pair makes a single claim in two halves. The companion showed **evolution without authoring a specification** — a new baseline derived, under governance, from a business problem. This paper shows **execution without originating behavior** — that baseline run by an interpreter that decides nothing. Construction, execution, and evolution stop being separate disciplines with separate tools. They become three governed compilations over one architecture, meeting at one immutable object. Call that whole — governed transformation above the protocol, declarative execution below it — **protocol-governed computing**. The term is introduced here deliberately, as the name for what the pair of papers jointly describes.

18. Implications

For runtimes, the consequence is a demotion that is also a clarification: the runtime becomes a commodity interpreter, and correctness moves upstream to the compiler. A team that adopts this architecture invests in its protocol and its compiler, and treats the runtime as replaceable infrastructure — which is what, under the Execution Partition, it is.

For verification and audit, behavior becomes checkable from the protocol and the trace rather than by instrumenting a live system. What a system will do is a property of an artifact one can read, and what it did is a property of a trace one can trust. Neither requires watching the system run and hoping to catch every path.

For portability and longevity, behavior outlives its engine. A protocol is an asset that survives the runtimes, languages, and hardware that execute it — a durability conventional systems cannot offer, because conventional systems keep some of their behavior in the very layers that turn over.

These implications are offered at the strength the work supports, and the evidence deserves the same plain statement of limits the companion gave its own. The architecture has been realized and exercised in a single reference runtime, built by a single practitioner who is also the architecture's designer. Within that limit the exercise is real: one runtime, with no domain knowledge, has executed two unrelated domains — a blockchain reference domain and an AI-governance domain — from a single compiled snapshot, with deterministic traces throughout; and the companion's validation stage runs on the same machinery. There is no second, independently built runtime yet, so runtime independence rests on the architectural argument rather than on a demonstrated pair — §19 makes closing that gap the first item of future work. That is enough to demonstrate the model and to make it worth adopting and testing more widely; it is an existence proof, not a controlled evaluation, and the paper claims nothing beyond it.

19. Future work

The most natural next step is to build a second, independent runtime for the same protocol and show that it produces identical behavior — turning runtime independence from an architectural claim into a demonstrated one, and yielding, in effect, a conformance suite for interpreters. The method is already published: the Runtime Conceptual Model describes multi-runtime certification — execute the same snapshot on both runtimes and compare trace semantics; the snapshot is the test suite, and the traces are the test results. What remains is to build the second interpreter and run it.

Beyond that lie a formal statement of the determinism theorem, with the invariants of §13 as axioms; distributed execution under the same Execution Partition, where the challenge is to preserve the partition across nodes; and the execution of the transformation loop itself as protocol, which is also the companion's future work — at which point construction, execution, and evolution are governed by one machinery all the way down. None of these is required for the model presented here. Each is a transformation of it, to be carried out and then executed through the same two-part architecture the pair describes.

20. Conclusion

The earlier work in this series established that behavior belongs in protocol rather than in implementation, and that a protocol's evolution can be governed rather than left to habit. The companion to this paper drew the first of those results to its end: software can evolve without a human authoring a specification. This paper draws the second to its end: software can execute without a runtime originating behavior.

The two results are one architecture, partitioning one thing — authority — at two altitudes. Authority over meaning belongs to the author; authority over behavior belongs to the compiler; the machine, at both altitudes, holds none. What is left to the runtime is realization, faithful and forgettable, of a behavior it did not and could not create. Programs once carried behavior; protocols now carry behavior; runtimes merely realize it.

> The executable protocol becomes the enduring execution artifact. The runtime becomes its deterministic interpreter.

Appendix A — Key Terms

Executable Protocol. An immutable, complete, executable description of system behavior that requires no behavioral inference during execution; the compiled Protocol Snapshot the runtime interprets.

Execution Closure. The property that everything required to execute is present in the protocol — no external knowledge, inference, or default is needed at run time. The execution-side counterpart of the completeness the companion's progressive semantic enrichment drives toward, and the condition that makes the Execution Partition enforceable.

Execution Partition. The rule that behavioral authority is partitioned absolutely: the compiler defines behavior; the runtime realizes it. Named, like the companion's Knowledge Partition, by what is partitioned. Law: the runtime may execute behavior but may never originate it. Corollary: nothing is decided at runtime that was not decided during compilation.

Protocol Interpreter. The runtime, understood as a walker of an already-decided structure. It interprets, schedules, dispatches, transitions state, and emits traces — and originates no behavior.

Capability. A governed unit of execution the runtime dispatches, fronted by a contract that declares its inputs, outputs, and enumerated outcomes. The contract is the entire interface between the runtime and the capability.

Contract. The declared interface of a capability — inputs, outputs, outcomes. The runtime knows the contract and nothing beneath it, which is why implementations are interchangeable.

Governed Store. A protocol-defined location and ownership for state. The runtime maintains stores but does not own them or decide their topology.

Trace. The runtime's output-only record of an execution: written as execution proceeds, never read back as input by any component. Because execution is deterministic, the trace is a faithful and complete account.

Determinism. The property that identical protocol, inputs, and initial state yield identical outputs and an identical trace — a consequence of the Execution Partition, not a feature added to the runtime.

Replay. Exact reproduction of a past execution by re-running the same protocol on the same inputs against the same initial state.

Attestation. The compiler-produced integrity record against which the runtime verifies, before any execution, that the protocol it holds is exactly what the compiler produced. On mismatch the runtime refuses to execute.

(Terms carried from the companion and the prior papers — Protocol Snapshot, Knowledge Partition, worker, compiler, admission, validation — retain their published definitions.)

Appendix B — Reference Implementation Notes

The architecture was realized and exercised in the open-source Protocol-Governed Systems reference implementation. The conceptual model is what endures; the implementation will change. Repository names, command surfaces, and artifact formats are confined to this appendix by intent, and the prior *Runtime Conceptual Model* paper is the implementation-level treatment of the reference interpreter. The reference implementation is available at https://github.com/bachipeachy/pgs_workspace.

Status at the time of writing: one reference interpreter, `pgs_runtime`, executing two domains — a blockchain reference domain and an AI-governance domain — from a single compiled snapshot. No second, independently built interpreter yet exists (see §19). Because the papers in this series are live simultaneously and describe progress at different moments, this status governs the notes below.

The runtime. The reference interpreter is the `pgs_runtime` engine. It is generic: it has no domain logic, reads only from the compiled snapshot, and writes only traces. It realizes the Behavior Partition directly — every behavior it performs is declared in the snapshot, and it originates none.

The nine execution concerns. In the reference implementation, protocol artifacts belong to named execution concerns that make the traversal of §7 concrete: transport ingress and egress (boundary normalization and projection), intent (admission), actor context (authority binding), workflow (the governed DAG), capability contract (the dispatched node), capability transform (pure computation) and capability side effect (bounded external interaction), and event (observability). Runtime bindings map contract declarations to concrete implementations — the mechanism behind the contract-only dispatch of §10. The pure/side-effecting distinction among capabilities, and the invariant that pure computations never perform side effects, are enforced at compile time.

State and stores. Storage topology is declared in structure and binding artifacts, never hardcoded in runtime code — the concrete form of §9's claim that state topology is protocol-governed. The runtime maintains stores at declared, absolute paths and performs only the declared transitions.

Traces. Each execution writes an append-only structured log, a human-readable summary, and a path visualization to a trace directory keyed by a deterministic trace identifier. Identical inputs produce an identical trace identifier — the reference implementation's expression of §8's determinism.

Worker independence at the execution boundary. As the companion exercised interchangeable workers at authoring time, the reference runtime exercises interchangeable implementations beneath a contract at execution time: an implementation may be replaced by any other that satisfies the same contract without the runtime's knowledge.

Appendix C — References

Ganti, B. (2026). *Protocol-Governed Systems: Conceptual Model*. DOI: https://doi.org/10.5281/zenodo.20300611

Ganti, B. (2026). *Protocol-Governed Systems: Compiler Conceptual Model*. DOI: https://doi.org/10.5281/zenodo.20471804

Ganti, B. (2026). *Protocol-Governed Systems: Runtime Conceptual Model*. DOI: https://doi.org/10.5281/zenodo.20478471

Ganti, B. (2026). *Protocol-Governed Systems: Architecture Inversion Concepts*. DOI: https://doi.org/10.5281/zenodo.20497732

Ganti, B. (2026). *Protocol-Governed Systems: Closed-Loop Governed Evolution* (v1). DOI: https://doi.org/10.5281/zenodo.21434335

Backus, J. (1978). *Can Programming Be Liberated from the von Neumann Style? A Functional Style and Its Algebra of Programs*. Communications of the ACM, 21(8), 613–641.

Codd, E. F. (1970). *A Relational Model of Data for Large Shared Data Banks*. Communications of the ACM, 13(6), 377–387.

Lindholm, T., Yellin, F., Bracha, G., & Buckley, A. (2014). *The Java Virtual Machine Specification, Java SE 8 Edition*. Addison-Wesley.

van der Aalst, W. M. P. (2013). *Business Process Management: A Comprehensive Survey*. ISRN Software Engineering, 2013, Article 507984.

Burns, B., Grant, B., Oppenheimer, D., Brewer, E., & Wilkes, J. (2016). *Borg, Omega, and Kubernetes*. ACM Queue, 14(1), 70–93.

Morris, K. (2016). *Infrastructure as Code: Managing Servers in the Cloud*. O'Reilly Media.

Lamport, L. (1994). *The Temporal Logic of Actions*. ACM Transactions on Programming Languages and Systems, 16(3), 872–923.

Appendix D — Worked Example: Executing a Chain Workflow

This appendix walks one real execution from beginning to end, so that every stage the body described in the abstract can be seen concretely. It is the deliberate counterpart to the companion's worked example, which *authored* a canonical **chain** subdomain — a serial, hash-linked, immutable ledger of finalized blocks. Here the same subdomain is *executed*. One paper authored the chain; this one runs it.

Throughout, watch for the one thing the architecture insists on: at no point does the runtime decide anything. It reads the protocol and realizes it.

The protocol. Before execution, the chain subdomain exists only as compiled protocol in the snapshot: a workflow for appending a finalized block, the capability contracts that workflow traverses, the storage the chain owns, and the events it emits. This protocol is immutable and complete — it exhibits execution closure. Nothing about the append behavior remains to be worked out at run time.

The input. A caller submits a finalized block to be appended, in the form the boundary expects. This is the only thing that enters from outside; everything the system will do in response is already fixed in the protocol.

Boundary normalization (ingress). The transport ingress concern normalizes the external input into the canonical internal form the protocol declares. No behavior is invented here; the normalization is a declared projection.

Admission (intent). The intent concern checks the normalized payload against the declared admission rules — is this a well-formed append request? — and returns an acknowledgment or a rejection. The runtime does not decide the criteria; it applies the declared ones.

Traversal (workflow). The workflow is now traversed as a governed graph. Its first capability verifies that the submitted block links to the current head of the chain. This capability reports one of its enumerated outcomes — the link is valid, or it is not — and that outcome selects the next node, according to routing the compiler fixed. If the link is invalid, the declared outcome routes to rejection; the runtime performs no branching logic of its own, and there is no path it can take that the protocol did not lay down. If the link is valid, traversal proceeds: a capability forms the chain record, a side-effecting capability appends it to the chain's governed store, and the declared lifecycle event is emitted — a value the protocol declares and a side-effecting capability records, which the runtime carries without interpreting. Each capability receives its inputs through declared references resolved against results the traversal already produced; none searches for them.

Boundary projection (egress). The transport egress concern formats the internal result for the caller — again a declared projection, not a decision.

The trace. As all of this proceeds, the runtime writes a trace: an append-only log of every step, its inputs, and its outcome. The trace is output only; nothing in this execution read it, and nothing later will treat it as input. Its identifier is deterministic — a function of the protocol and the inputs.

Replay, and the second run. Determinism is a claim about protocol, inputs, and initial state together, and the worked example shows why the third term matters. Submit the same block a second time, and the execution does *not* repeat the first — deliberately. The first append moved the head of the chain; the same block no longer extends it. The second run therefore takes a different declared path: the link check reports its negative outcome, and the routing fixed at compile time carries it to the declared rejection. Nothing about this is improvised — the second run is exactly as deterministic as the first, and running it a third time reproduces the second identically. This is the architecture handling duplicates by declaration rather than by runtime cleverness. Replay in the strict sense — exact reproduction of the first execution — is obtained by re-running the same protocol on the same input against the same initial state: the same traversal, the same outcomes, the same store transition, the same trace, under the same identifier. Both demonstrations matter. The second run shows that the negative path is as declared as the positive one; strict replay shows that, with the state held fixed, nothing varies at all.

At no point in this walkthrough did the runtime determine behavior. It normalized by declaration, admitted by declaration, traversed by declared outcomes, transitioned declared state, and recorded what it did. Everything the system did was in the protocol before the run began; the runtime realized it and added nothing. That is the architecture, in one concrete execution — and the same chain the companion authored, now shown running under the law this paper defends.